David Gamez (david.gameza@urv.cat)

Course	Embedded and Mobile Applications
Document	AME26_27_EMBEDDED_MANDATORY_EXERCISE.pdf
Date	15/10/2026
Description	Mandatory exercise for the embedded part of the subject
Scope	Mandatory

Statement.

You must create an embedded application monitor and control the environment of a managed using the following tasks.

a) Measure room **temperature** and **ambient light**.

   (You can use specifications and libraries done in the optional exercises 1 and 2)

b) Allow the user to configure a temperature threshold (up and down). Below or up this the alarm shall be triggered.

c) Detect normal, warning and alarm conditions using a **finite-state machine**.

d) Acknowledge alarms with an **interrupt-driven push button**.

e) Generate an **audible state-dependent** warning.

f) Control artificial **room illumination using PWM**.

g) Filter sensor measurements and apply hysteresis.

h) Experimentally validate timing and PWM behavior.

i) Show the date using the **RGB LCD.**

Required equipment.

- ST NUCLEO-F767ZI development board
- Temperature Sensor
- Light Sensor
- Rotary Angle Sensor
- Button
- Buzzer
- LED or another LED connected through a suitable output stage
- RGB LCD

Embedded and Mobile Applications

Operation.

The system shall sense the temperature and the light. The ambient temperature shall be between two thresholds, one in the bottom and another in the top, these thresholds shall be configurable using the rotary angle.

Using a finite state machine the program shall determine if the system is in NORMAL, WARNING, ALARM and AKNOWLEDGED state. Following a summary of the different state system behavior:

State	Required behavior	State transition
NORMAL	Alarm silent. Normal visual indication. Continue monitoring sensors.	
WARNING	Visual warning. Slow audible warning. Continue monitoring sensors.	
ALARM	Alarm indication. Fast audible alarm. User may acknowledge the alarm.	
ACKNOWLEDGED	Abnormal condition remains visible, audible alarm is silenced, monitoring continues.	

(Task) Please define the state transition thresholds in the top table, for instance: if the measured temperature is below or up the threshold changes from NORMAL to WARNING if the measured temperature is below or up the threshold for 3 degrees change to ALARM state.

Please be aware that to **change between the status you will need a filter like hysteresis** that we have explained in the Optional exercise 2.

To change to ACKNOWLEDGED you must use an Interrupt and the button USER push-button is connected to PC13.

To change the thresholds, you can use the button, for instance if the button is not pressed you can change the low threshold if is pressed the upper threshold.

Interrupts.

In MBedOS the interrupts are working with the type InterruptIn. How is working.

You need to use the constructor to declare a variable, in the constructor you must put the pin name, for instance:

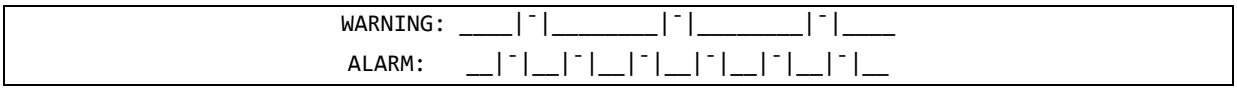
```
InterruptIn btnI(Button1);
```

And in the `main()` function, outside the infinite loop you must declare the function to call when the signal is going from 0 to 5V (rise) or from 5V to 0 V (fall), for instance:

```
btnI.fall(&function);
```

When the system is in ALARM state must use an audible alarm. You can find the specification of usage in the campus virtual.

The audible alarm shall sound using this patterns:



At the same time, you must control the illumination, in order to do it the system needs to sense the light as we did in the optional exercise 2 and actuate in a LED, for this you need to use the LED with different intensities you must use the PWM technique (explained below). The rule is: **system shall have always 100 % of the light**, for instance, if you are sensing 80 % of the light with the sensor the LED shall bright at 20 %.

Data log.

All data that you consider relevant, for instance the current temperature, thresholds, …. Must be shown in the LCD display, library and API are available in the CampusVirtual. Consider that I'll check the correct functionality without connecting the board to a computer, so please show all relevant data in the display.

To control the output led you should use a PWM (Pulse Width Modulation).

PWM (Pulse Width Modulation).

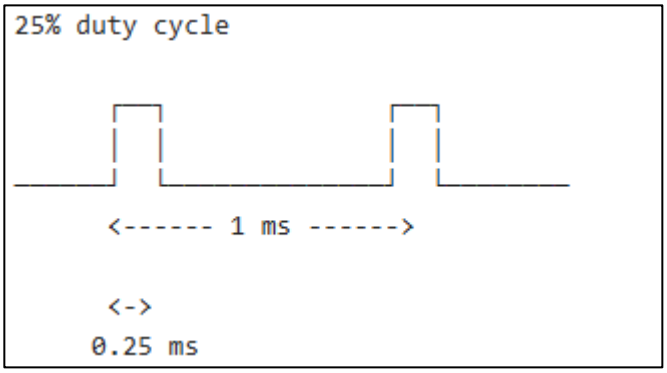
PWM (Pulse Width Modulation) generates a periodic digital signal that switches between HIGH and LOW.

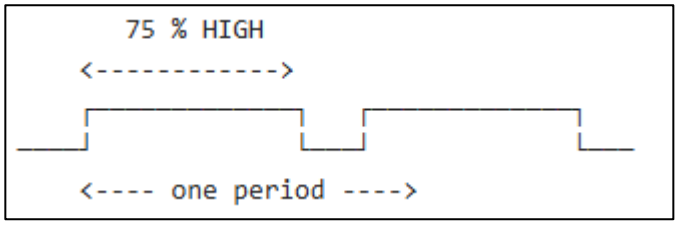
Two parameters define the signal:

- **Period** T: duration of one complete PWM cycle.
- **Duty cycle** D: percentage of the period for which the signal remains HIGH.

For example, a 1 kHz PWM has:

$$T = \frac{1}{f} = \frac{1}{1000} = 1 \text{ ms}$$





In mbedOS you must use PwmOut class.

For usage you must declare the variable with the pin number as a variable. For instance:

```
PwmOut led(D3);
```

In the main(), before the infinite loop you must set the period:

```
led.period_ms(10); // Period = 10 ms
```

You can change the duty cycle using the following function:

```
led.write(0.5f); // Duty cycle = 50 %
```

Delivery of exercise.

This exercise must be delivered at the campusvirtual inside the task enabled for this purpose. The maximum day of the delivery is the 20th of November 2026 at the 23:55 h in the proper space for the delivery

Important notes for the delivery.

Your delivery needs to be named:
AME26_27_DEVELOPER1_DEVELOPER2_MANDATIRY_EXERCISE1.zip